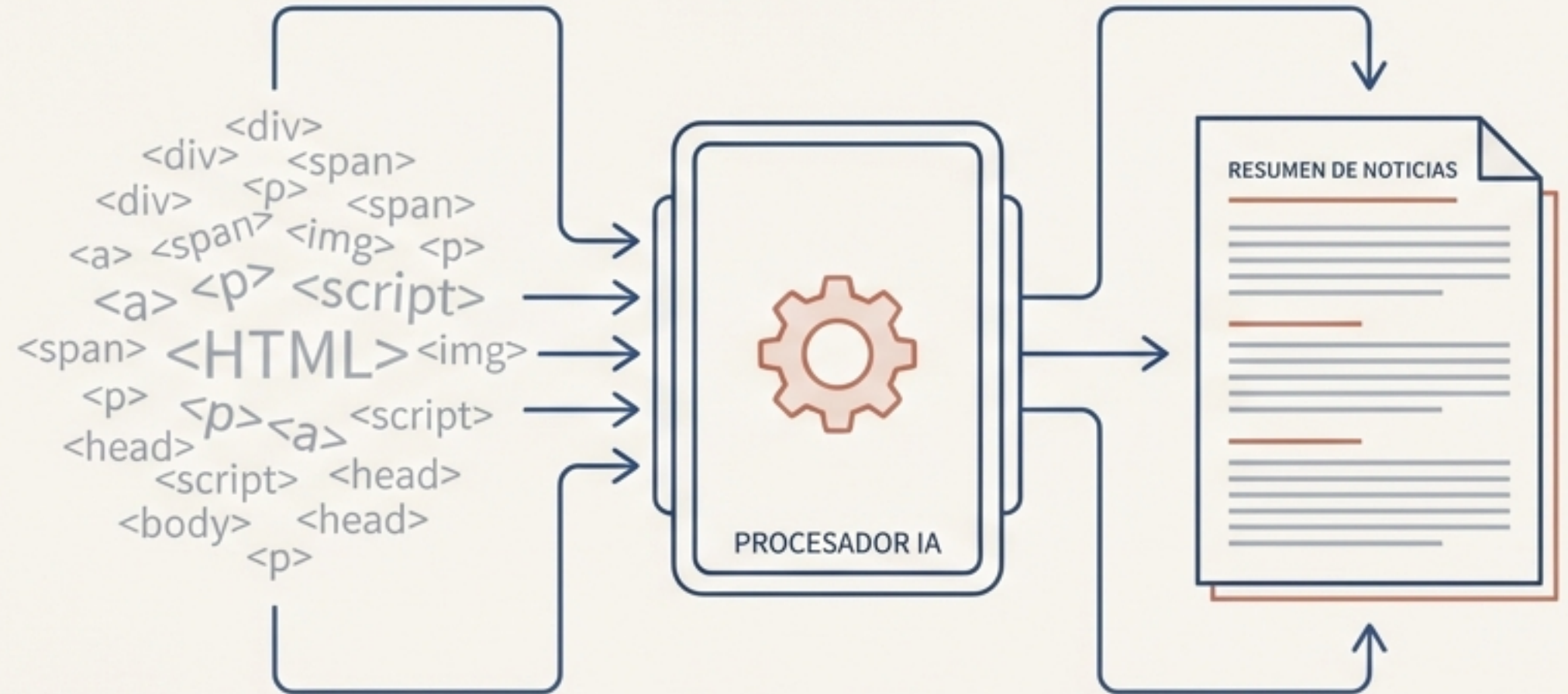


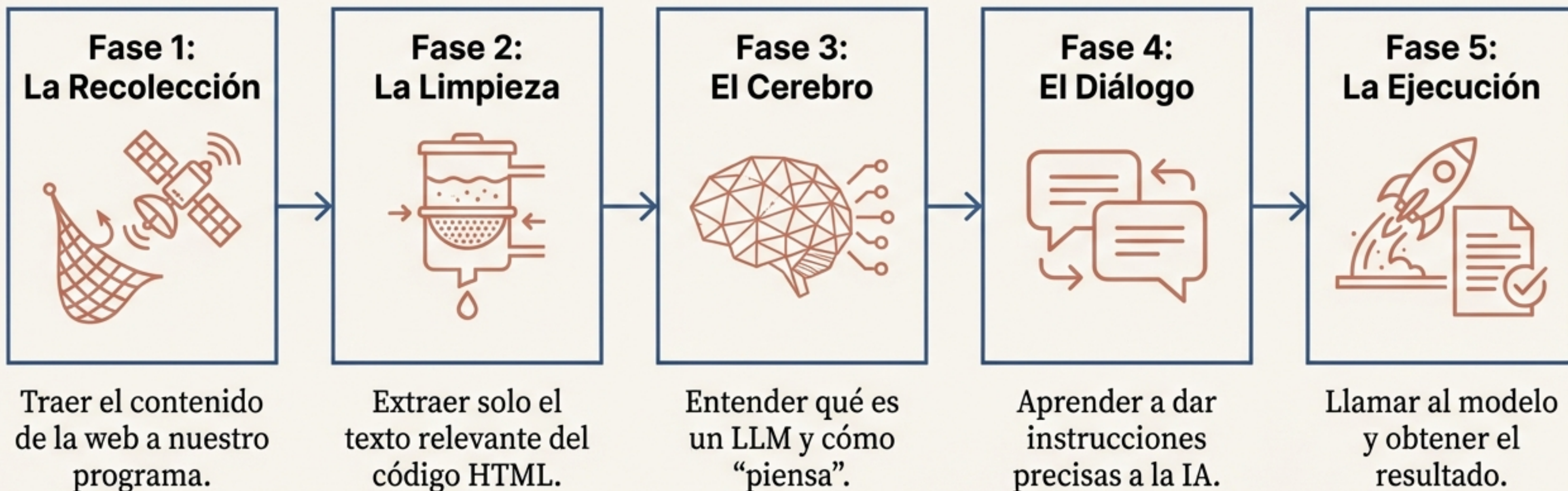
De la Web al Resumen: Anatomía de un Lector con IA

- Transformaremos el caos de internet en resúmenes claros y útiles.
- Construiremos un programa que lee y entiende noticias deportivas por nosotros.
- Aprenderemos la teoría detrás de cada línea de código que escribimos.
- De la práctica a los fundamentos: el viaje de un desarrollador profesional.



Analogía: Imagina que tienes un asistente personal que cada mañana lee todos los periódicos deportivos por ti y te entrega un informe de una sola página con lo más importante. Hoy, vamos a construir ese asistente.

Nuestro Plan de Vuelo: Las 5 Fases Clave del Proceso

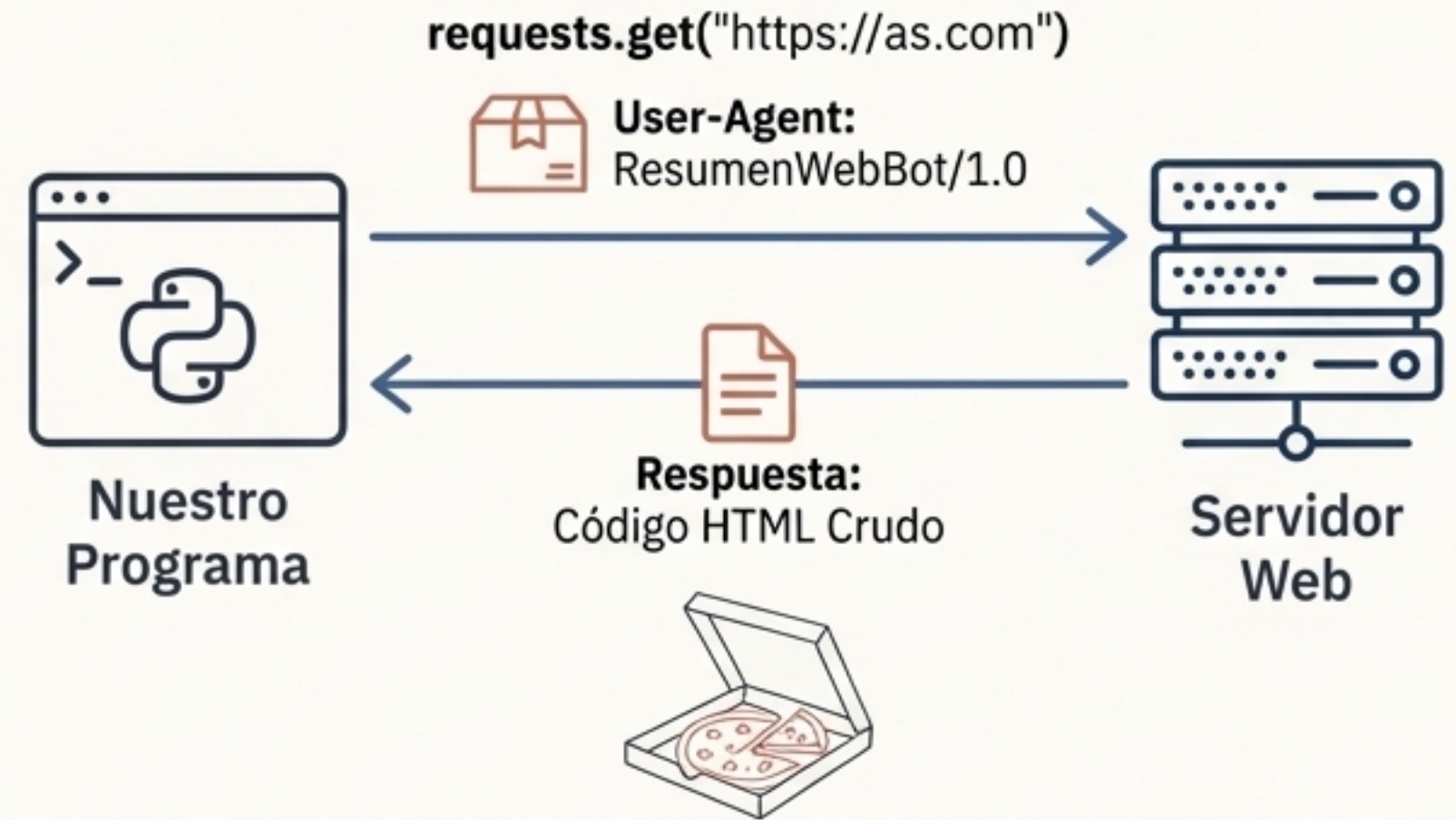


Analogía: Es como cocinar una receta. Primero, compras los ingredientes (recolección). Luego, los lavas y preparas (limpieza). Usas el horno (el cerebro). Sigues las instrucciones (el diálogo). Finalmente, sirves el plato (la ejecución).

Fase 1 | La Recolección

Hablando el Idioma de la Web
con `requests`

- Usamos la librería `requests` para actuar como un navegador web.
- Con `requests.get(url)` realizamos una petición HTTP `GET` para solicitar el contenido.
- El `User-Agent` nos identifica para evitar ser bloqueados.
- Recibimos el código HTML crudo, la 'materia prima' de la página.

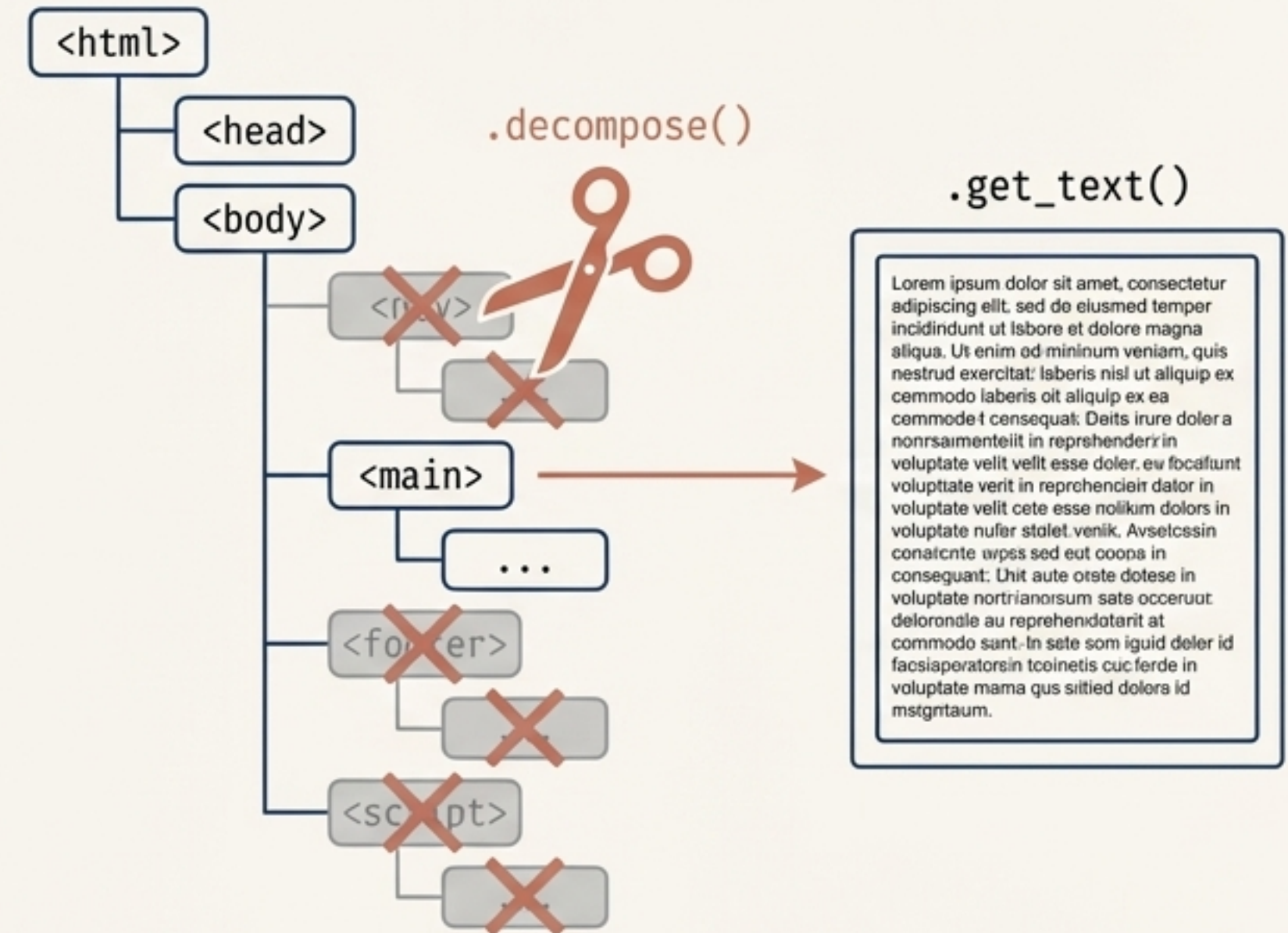


```
# Realizar la petición
response = requests.get(
    url,
    timeout=20,
    headers={"User-Agent": "ResumenWebBot/1.0"}
)
# Verificar si hubo errores (ej. 404)
response.raise_for_status()
```


Fase 2 | La Limpieza

Esculpiendo Datos con `BeautifulSoup`

- El HTML es un "árbol" de etiquetas; necesitamos navegarlo para encontrar lo útil.
- `BeautifulSoup` convierte el texto HTML en un objeto de Python que podemos manipular.
- Eliminamos el "ruido" con `.decompose()`: etiquetas como `<nav>`, `<footer>` o `<script>`.
- Extraemos el texto limpio y estructurado con `.get_text()`.



```
soup = BeautifulSoup(response.content, 'html.parser')

# Eliminar "ruido"
for tag in soup.body(["script", "style", "footer", "nav", "aside"]):
    tag.decompose()

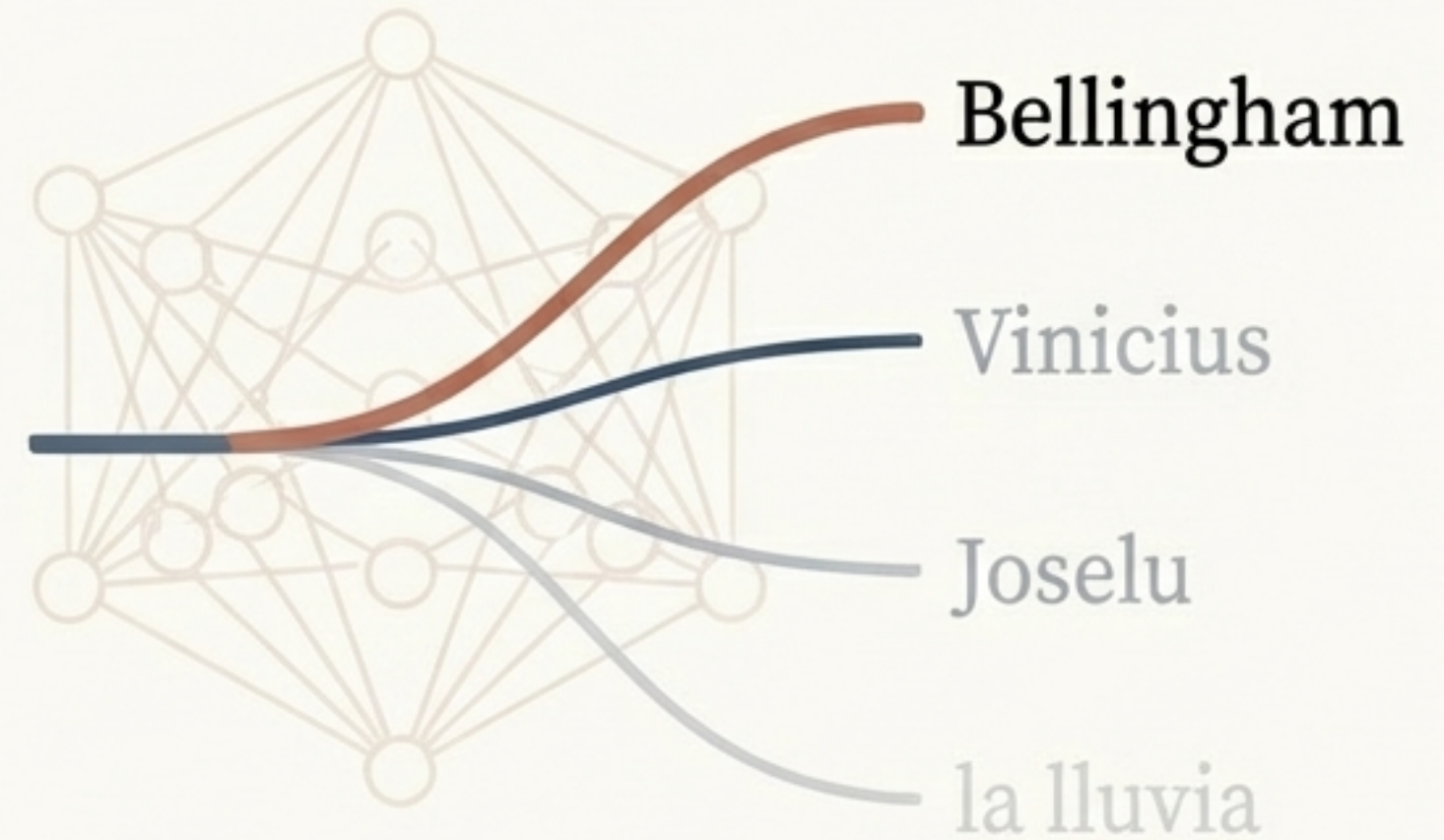
# Extraer solo el texto
texto_limpio = soup.body.get_text(separator="\n", strip=True)
```


Fase 3 | El Cerebro

¿Qué es un Modelo de Lenguaje (LLM)?

- Un LLM (Large Language Model) es una red neuronal entrenada con enormes cantidades de texto.
- No 'entiende' como un humano, sino que es experto en predecir la siguiente palabra más probable.
- Esta habilidad para predecir secuencias le permite resumir, traducir y generar texto coherente.
- En nuestra práctica usamos `llama3.2` (local) o `gpt-5-nano` (nube).

El Real Madrid ganó el partido gracias a un gol de...

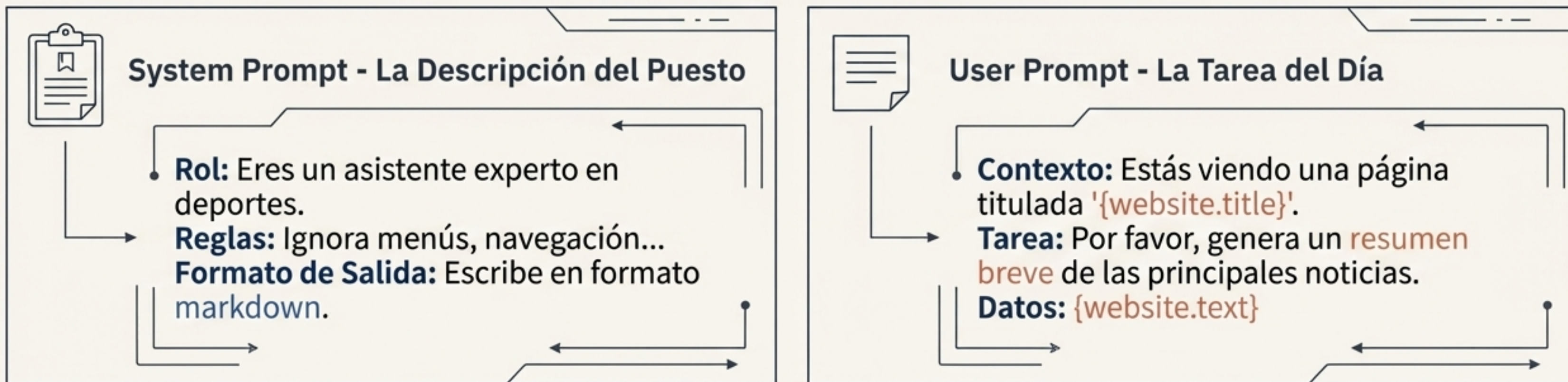


Analogía: Un LLM es como un bibliotecario que ha leído todos los libros del mundo. No tiene opiniones propias, pero si le das el inicio de una frase, puede completarla con una precisión asombrosa basándose en los patrones que ha visto millones de veces.

Fase 4 | El Diálogo

El Arte de la Ingeniería de Prompts

- Un 'prompt' es la instrucción que le damos a la IA para guiar su respuesta.
- **System Prompt:** Define el rol y las reglas del asistente ("Eres un experto en deportes...").
- **User Prompt:** Contiene la tarea específica y los datos (Resume este texto...).
- La calidad del prompt determina la calidad del resumen.



```
messages = [  
    {"role": "system", "content": system_prompt},  
    {"role": "user", "content": user_prompt}  
]
```


Fase 5 | La Ejecución: Dos Caminos para la IA

Ollama (Open Source)



- Se ejecuta en tu propio ordenador.
- Mayor privacidad y control.
- Sin costes por uso.

```
# Llamada a un servicio local
response = ollama.chat(
    model="llama3.2",
    messages=messages
)
```

OpenAI (Cloud API)



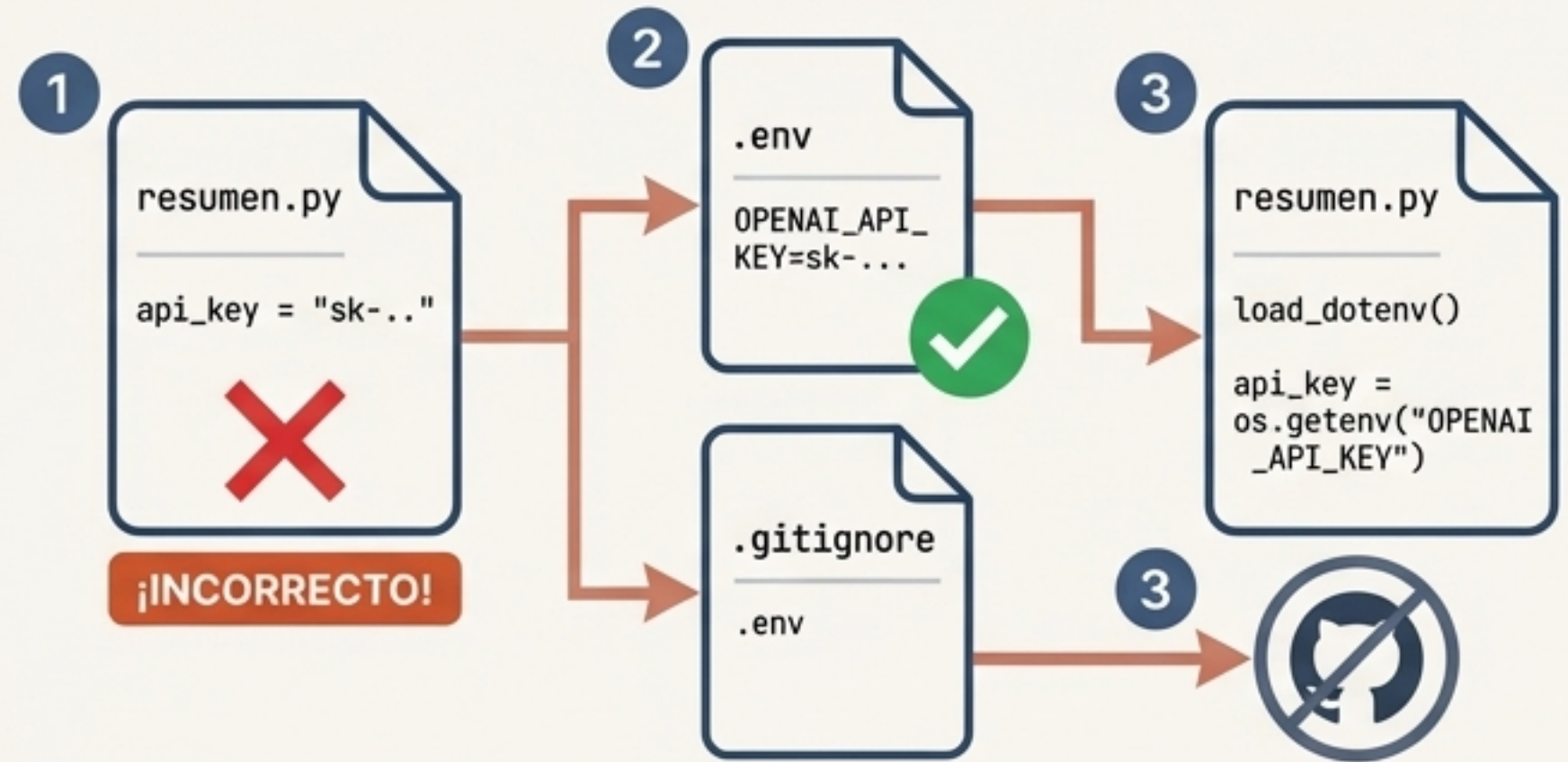
- Accedes a través de internet (API).
- Gestionado por un tercero.
- Modelo de pago por uso.

```
# Llamada a un servidor remoto
response = client.chat.completions.create(
    model="gpt-5-nano",
    messages=messages
)
```


Buenas Prácticas (I)

Gestión de Secretos con `dotenv`

- Las claves de API (`API Keys`) son como contraseñas. ¡Nunca deben estar en el código!
- El archivo `.env` almacena variables de entorno de forma local y segura.
- `.env` se excluye del control de versiones (Git) para no exponer las claves.
- La librería `dotenv` carga estas variables para que nuestro programa pueda usarlas.



```
# Instala la librería dotenv
pip install python-dotenv

# En tu código:
import os
from dotenv import load_dotenv

load_dotenv() # Carga las variables de entorno desde .env

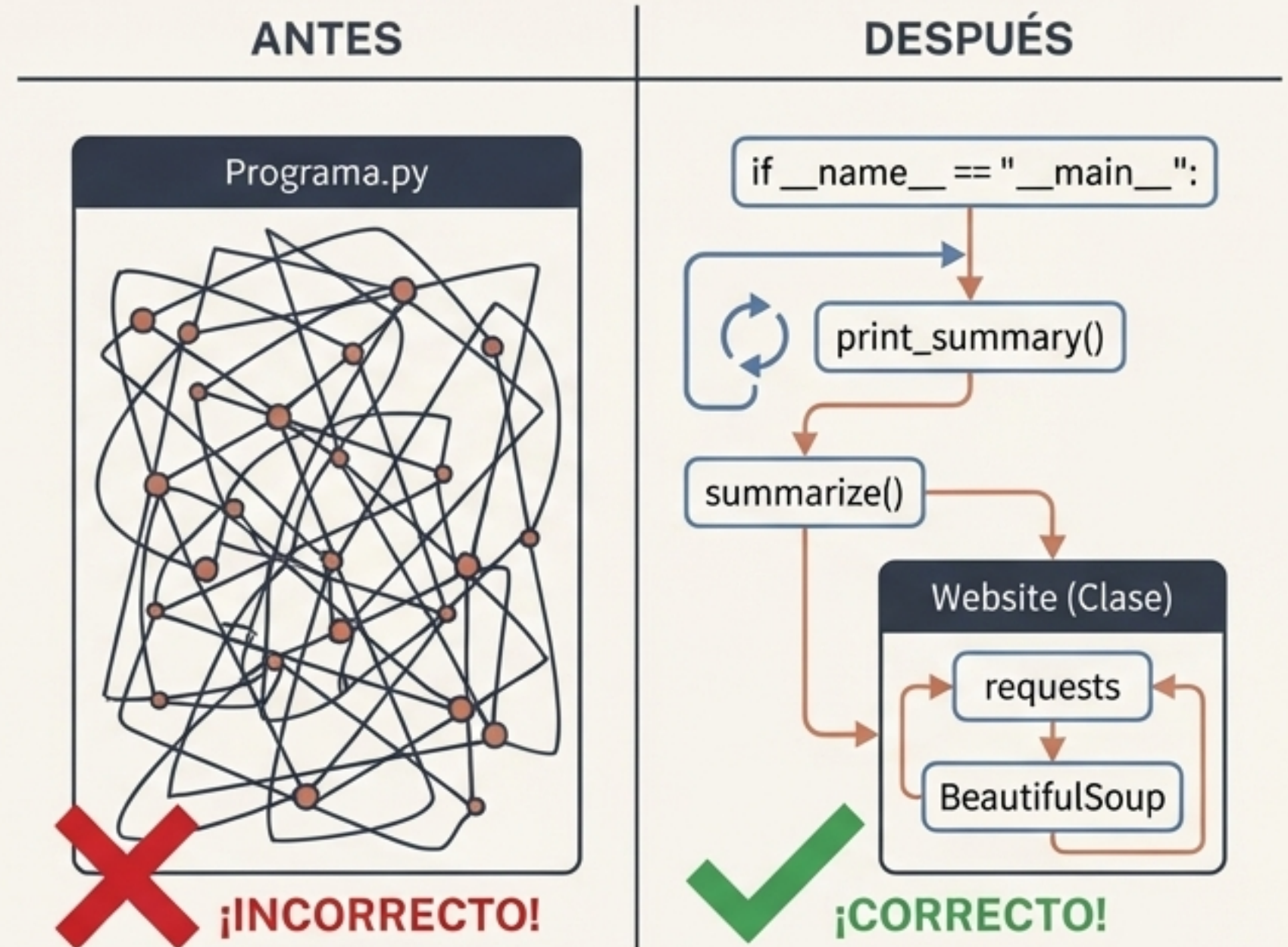
api_key = os.getenv("OPENAI_API_KEY")
# Ahora puedes usar la variable api_key de forma segura.
```

Analogía: Tu clave de API es la llave de tu casa. No la dejas pegada en la puerta (el código fuente). La guardas en tu bolsillo (el archivo `.env`).

Buenas Prácticas (II)

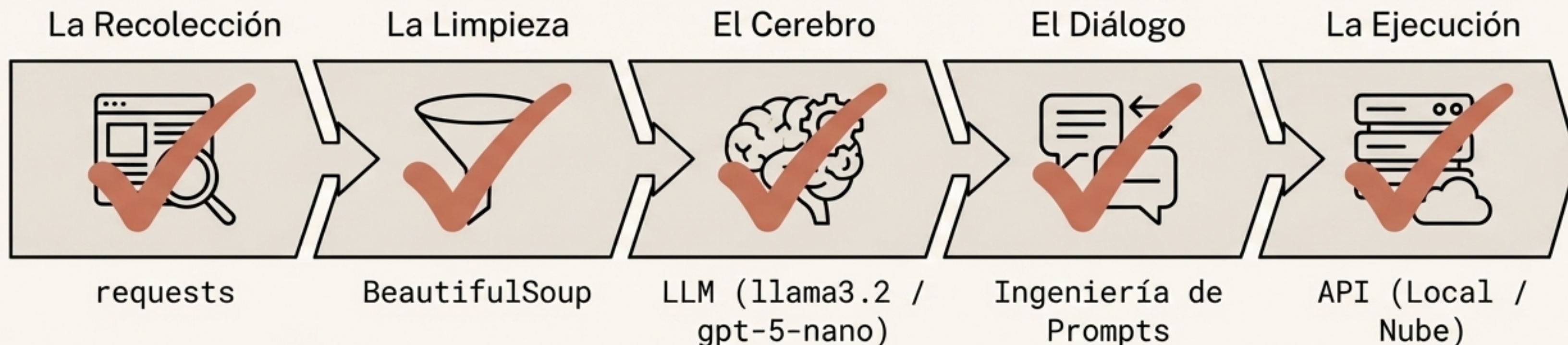
Orquestando el Código

- La clase `Website` agrupa la lógica de descarga y limpieza en un solo lugar.
- Cada función tiene una única responsabilidad (SRP - Single Responsibility Principle).
- El bloque `if \_\_name\_\_ == "\_\_main\_\_":` es el punto de entrada de nuestro script.
- Esta estructura hace el código más legible, reutilizable y fácil de mantener.



Analogía: Un coche bien diseñado. Como conductor, solo usas el volante y los pedales (la interfaz pública), no necesitas conocer la mecánica interna de cada componente (las funciones y clases) para que todo funcione.

Conclusiones: El Viaje del Dato Completado



- Hemos construido un *pipeline* completo: de la URL al resumen inteligente.
- Comprendemos el rol de cada herramienta: `requests` (el mensajero), `BeautifulSoup` (el escultor) y el LLM (el cerebro).
- Sabemos que la ingeniería de *prompts* es la clave para dirigir a la IA.
- Podemos decidir estratégicamente entre un modelo local (control) y uno en la nube (conveniencia).

Errores Comunes: Programando a la Defensiva



Error de Red

La web falla o no responde
(`requests.exceptions.RequestException`).

```
try:  
    print_summary(u)  
except Exception as e:  
    print(f"[ERROR] {u}: {e}")
```



Cambios en HTML

La web cambia su diseño y el scraper se rompe.

Actualizar los selectores en
`.decompose()` o usar métodos de
extracción más robustos.



Límites de Contexto

Enviamos demasiado texto al modelo
y da un error.

Truncar el texto para asegurar que cabe
en la ventana de contexto del modelo.

```
self.text = self.text[:12000]
```



Errores de API

El servicio de IA está caído o la clave
API es incorrecta.

Usar `try/except` para capturar
errores de la API y mostrar un
mensaje claro al usuario.